Contents

Chapter 5: Agentic Workflows

The defining characteristic of an AI agent is not that it uses an LLM. It is that the LLM participates in a **closed-loop control system**.

A conventional LLM application looks like this:

```
Prompt
   ↓
 LLM
   ↓
Response
```

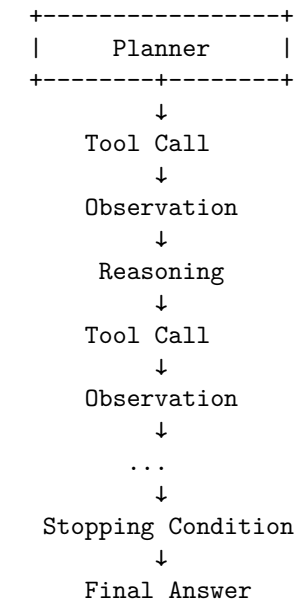
The model receives context, computes a response, and stops.

The next step is to give the model access to an external capability:

```
Prompt
   ↓
 LLM
   ↓
 Tool
   ↓
Response
```

Now the model can affect the outside world. It can retrieve a document, execute a query, invoke an API, manipulate a file, or perform some other operation.

But the most interesting systems go further:



The system is no longer simply generating an answer. It is **acting, observing the consequences of its actions, updating its state, and deciding what to do next**.

That is the essence of an agentic workflow.

For an engineer, however, the important question is not:

> "How do I build an agent?"

It is:

> **"How do I build a bounded, observable, recoverable control loop in which a probabilistic model can safely perform useful work?"**

That distinction matters. Agents are easy to prototype. Reliable agents are systems engineering.

---

1. The Evolution from LLM Calls to Agents

There is a natural progression in AI application architecture.

1.1 Stage 1: Prompt → LLM

The simplest application is a pure function:

```
response = llm(prompt)
```

Conceptually:

$$y = f_\theta(x)$$

where x is the prompt, f_θ is the model, and y is the generated output.

This architecture works well when the task can be completed entirely from the information supplied to the model.

Examples include:

- summarization
- classification
- rewriting
- extraction
- translation
- code generation
- question answering over supplied context

But the model is fundamentally isolated from the world.

It cannot discover information that was not provided to it. It cannot inspect the current state of a system. It cannot take actions.

---

2. Stage 2: LLM + Tools

The next step is tool calling.

```
User
 ↓
LLM
 ↓
Tool Call
 ↓
Tool
 ↓
Result
 ↓
LLM
 ↓
Response
```

Suppose the user asks:

> What is the weather in Portland?

The model might produce a structured request:

```
{
  "tool": "get_weather",
  "arguments": {
    "location": "Portland, Oregon"
  }
}
```

The application executes the tool:

```
result = get_weather("Portland, Oregon")
```

and feeds the result back to the model.

The model can now ground its response in external information.

This is already a major architectural change.

The LLM is no longer merely a generator. It is becoming a **decision-making component embedded inside a larger software system**.

---

3. Tool Calling Is an API Contract

Tool calling should be thought of as an API boundary, not as a magical capability of the model.

A tool has:

```
name
description
input schema
execution semantics
output schema
failure semantics
permissions
```

For example:

```
{
  "name": "search_documents",
  "description": "Search the internal document corpus",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string"
      },
      "limit": {
        "type": "integer",
        "minimum": 1,
        "maximum": 20
      }
    },
    "required": ["query"]
  }
}
```

The schema is important because the model is probabilistic while the tool interface should be deterministic.

The boundary therefore looks like:

```
Probabilistic
     LLM
      |
      | structured request
      v
Deterministic
     Tool
      |
```

```
      | structured result
      v
Probabilistic
     LLM
```

This separation is one of the fundamental design patterns of reliable AI systems.

The model decides **what it wants to do**.

The application decides **whether it is allowed to do it**.

The tool determines **what actually happened**.

---

4. Agents vs. Workflows

The word *agent* is often used too loosely.

Not every multi-step AI system is an agent.

Consider this workflow:

```
Input
 ↓
Extract entities
 ↓
Retrieve documents
 ↓
Summarize documents
 ↓
Generate report
```

There may be several LLM calls, but the execution graph is predetermined.

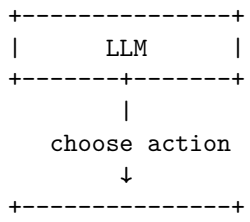
This is a **workflow**.

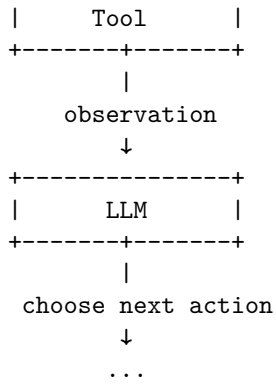
A workflow can be represented as:

$$S_0 \to S_1 \to S_2 \to S_3$$

The developer determines the control flow.

An agent is different:





The model participates in determining the next transition.

Formally, a workflow has a largely predetermined transition function:

$$s_{t+1} = F(s_t)$$

An agent introduces model-mediated decision making:

$$a_t \sim \pi_\theta(a \mid s_t)$$

followed by an environmental transition:

$$s_{t+1} = T(s_t, a_t)$$

where:

- s_t is the current state,
- a_t is an action,
- π_θ is the model's policy,
- T represents the external environment.

This is much closer to a control system than to a conventional function call.

---

5. The Agent Loop

A minimal agent can be expressed with surprisingly little code.

```
state = initial_state()

while not stopping_condition(state):

    decision = llm(state)
```

```
    if decision.type == "tool_call":
        observation = execute_tool(decision)
        state = update_state(state, observation)

    elif decision.type == "final":
        return decision.answer

    else:
        raise InvalidModelOutput()
```

The apparent simplicity is deceptive.

Every line hides an engineering problem.

`llm(state)` What exactly is state?

How much history should be supplied?

What if the model emits invalid arguments?

`execute_tool(decision)` Is the tool safe?

Does the user have permission?

Can it mutate data?

Can it spend money?

Can it leak private information?

`update_state(...)` What information should be retained?

What should be discarded?

How large can the state become?

`stopping_condition(...)` What prevents infinite loops?

What happens when the model repeatedly performs the wrong action?

This is where agent engineering begins.

---

6. State Is the Agent's Memory

An agent needs state.

A useful abstraction is:

$$S_t = (G, H_t, O_t, M_t, P)$$

where:

- G = user goal
- H_t = interaction history
- O_t = observations obtained so far
- M_t = intermediate memory
- P = system policies and permissions

A simple implementation might contain:

```
state = {
    "goal": user_request,
    "messages": [],
    "observations": [],
    "artifacts": [],
    "step_count": 0,
    "budget_remaining": 20,
}
```

State should not be confused with conversation history.

Conversation history is only one component.

A production agent may maintain:

```
Agent State
+-- User goal
+-- Conversation history
+-- Tool results
+-- Retrieved documents
+-- Working memory
+-- Intermediate conclusions
+-- Generated artifacts
+-- Authentication context
+-- Permissions
+-- Cost budget
+-- Time budget
+-- Step count
+-- Failure history
```

The distinction becomes important as agents become long-running.

---

7. Observation Is Different from Reasoning

A common architectural mistake is to treat tool output as though it were merely another prompt.

It is useful to distinguish three layers:

```
Action
  ↓
Environment
  ↓
Observation
  ↓
Interpretation
  ↓
Decision
```

Suppose an agent runs:

```
search("Apple M5 GPU architecture")
```

The search engine returns ten documents.

Those documents are **observations**.

The model's interpretation of those documents is **reasoning**.

The next search query is an **action**.

Keeping these concepts distinct makes the system easier to debug.

A useful trace therefore looks like:

```
Step 1
Action:
    search("M5 GPU architecture")

Observation:
    10 search results

Reasoning:
    Results suggest the relevant information is
    contained in Apple's architecture documentation.

Step 2
Action:
    retrieve(document_3)

Observation:
    8,200 tokens of document text

Reasoning:
    The document describes the GPU architecture
    but does not contain the requested benchmark.

Step 3
Action:
```

```
    search("M5 GPU benchmark ...")
...
```

This trace is dramatically more useful than simply logging the final answer.

---

8. Planning

Planning is the process of deciding what actions are required to accomplish the goal.

There are several levels of planning.

Reactive planning

The model decides one action at a time:

```
observe
  ↓
decide
  ↓
act
  ↓
observe
```

This is often sufficient for simple tasks.

Explicit planning

The model first constructs a plan:

```
Goal
 ↓
Plan
 +-- Search for X
 +-- Retrieve Y
 +-- Compare X and Y
 +-- Produce report
```

Then the system executes it.

Hierarchical planning

Complex goals can be decomposed:

```
Research topic
|
+-- Understand architecture
|   +-- Find primary source
```

```
|   +-- Extract specifications
|
+-- Evaluate performance
|   +-- Find benchmarks
|   +-- Normalize results
|
+-- Produce conclusion
```

The more autonomous the system becomes, the more important it is to distinguish:

planning from **execution**.

A planner should not necessarily have permission to execute every action it proposes.

---

9. Reflection

Reflection introduces another reasoning step after an action or after a completed task.

For example:

```
Generate answer
      ↓
Critique answer
      ↓
Identify deficiencies
      ↓
Revise answer
```

An agent might ask itself:

- Did I actually answer the question?
- Are the claims supported?
- Did I use primary sources?
- Is important information missing?
- Did any tool call fail?
- Should I gather more evidence?

Reflection can improve quality, but it has a cost.

If every action is followed by multiple additional LLM calls, latency and cost increase rapidly.

More importantly, reflection is not automatically reliable.

An LLM judging another LLM output is still a probabilistic process.

Therefore reflection should be treated as another component to evaluate—not as an oracle.

---

10. Loops and the Problem of Non-Termination

The agent loop introduces a problem that ordinary request/response applications largely avoid:

the system may never stop.

Consider:

```
while True:
    action = llm(state)
    result = execute(action)
    state = update(state, result)
```

There is no guarantee that the model will ever produce a final answer.

It could:

```
search
search
search
search
search
...
```

or:

```
retrieve A
retrieve B
retrieve A
retrieve B
...
```

A production system therefore needs explicit stopping conditions.

---

11. Stopping Conditions

A robust agent usually has multiple termination mechanisms.

Goal completion

```
if state.goal_complete:
    stop()
```

Maximum steps

```
if state.step_count >= MAX_STEPS:
    stop()
```

Token budget

```
if state.tokens_used >= MAX_TOKENS:
    stop()
```

Financial budget

```
if state.cost >= MAX_COST:
    stop()
```

Time budget

```
if elapsed_time >= MAX_RUNTIME:
    stop()
```

Repeated-state detection

```
if state_hash in previous_states:
    stop()
```

Tool failure threshold

```
if consecutive_failures >= MAX_FAILURES:
    stop()
```

A particularly important principle is:

> **Never rely on the model alone to decide when the system should stop.**

The model can propose termination.

The runtime should enforce it.

---

12. Retries

Tools fail.

Networks fail.

APIs return errors.

Authentication expires.

Search results are malformed.

Models generate invalid arguments.

An agent therefore needs failure recovery.

The simplest strategy is retry:

```
for attempt in range(MAX_RETRIES):
    try:
        return execute_tool(call)
    except TransientError:
        backoff(attempt)
```

But retries should distinguish between failure classes.

```
Tool Failure
|
+-- Transient
|   +-- Retry
|
+-- Invalid Input
|   +-- Repair request
|
+-- Authentication
|   +-- Re-authenticate / escalate
|
+-- Permission
|   +-- Deny
|
+-- Rate Limit
|   +-- Backoff
|
+-- Permanent Failure
    +-- Alternative strategy / terminate
```

Blindly retrying every error is dangerous.

If a tool rejects an operation because the user lacks permission, retrying ten times does not make the operation more valid.

---

13. Failure Recovery

A mature agent should be designed around failure rather than success.

Consider a research agent:

```
Search
 ↓
Retrieve
 ↓
```

```
Analyze
 ↓
Write report
```

What happens if retrieval fails?

A naïve agent might hallucinate the missing information.

A better architecture explicitly represents uncertainty:

```
Retrieve
   ↓
FAILED
   ↓
Retry
   ↓
FAILED
   ↓
Alternative source
   ↓
FAILED
   ↓
Report limitation
```

The final report should be able to say:

> The requested information could not be verified from the available sources.

That is a successful system behavior.

The objective is not:

> always produce an answer.

The objective is:

> produce the best answer justified by the available evidence, while accurately representing uncertainty and failure.

---

14. Delegation

As agent systems become more sophisticated, one agent can delegate subtasks to other agents or specialized components.

For example:

```
                Research Agent
                      |
          +----------+----------+
```

```
        ↓          ↓          ↓
Search Agent  Data Agent  Writing Agent
```

The research agent might delegate:

`"Find primary sources"`

to one component and:

`"Extract quantitative results"`

to another.

Delegation introduces another distributed-systems problem.

Now the system must manage:

- task boundaries
- context transfer
- authentication
- budgets
- result validation
- failure propagation
- duplicate work
- conflicting results

Delegation should therefore be introduced only when it provides a meaningful architectural advantage.

Multiple agents do not automatically produce a better system.

Often, a deterministic workflow with one capable model is simpler and more reliable.

---

15. Permissions: The Agent Is Not the User

One of the most important principles in agentic system design is:

> **Never confuse model intent with authorization.**

Suppose the model produces:

```
{
  "tool": "delete_database",
  "arguments": {
    "database": "production"
  }
}
```

The fact that the model selected the tool does not mean it should be executed.

The architecture should instead look like:

```
LLM
 ↓
Proposed Action
 ↓
Policy Engine
 ↓
Authorization
 ↓
Tool
```

The policy layer can enforce rules such as:

```
read documents          → allowed
search web              → allowed
modify local file       → allowed
send email              → confirmation required
delete data             → prohibited
transfer money          → confirmation required
access private data     → authorization required
```

The model should never be the ultimate authority over its own permissions.

This is analogous to traditional security architecture:

untrusted input must not directly control privileged operations.

---

16. Human-in-the-Loop

For high-impact actions, the system may require explicit human approval.

```
Agent proposes action
        ↓
Risk assessment
        ↓
Low risk? ----- Yes ---→ Execute
        |
        No
        ↓
Human approval
        ↓
Execute / Reject
```

This is especially appropriate for:

- financial transactions
- destructive operations
- external communications
- production deployments

- legal commitments
- changes to security controls

The approval boundary should be determined by **action risk**, not by how sophisticated the model appears.

---

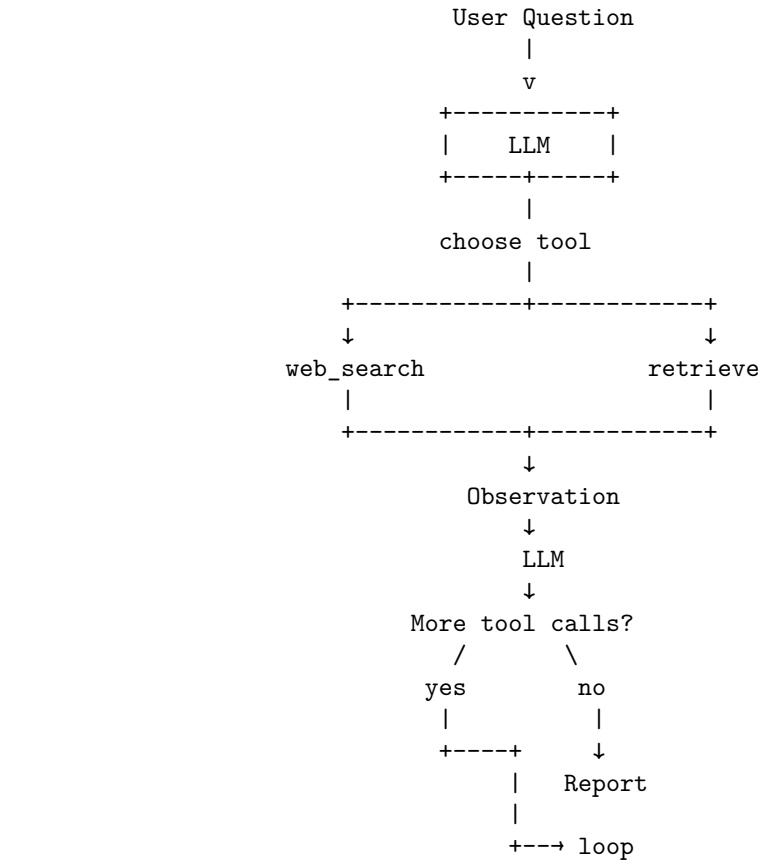
17. A Small Research Agent

Now build a small agent that performs a useful end-to-end task.

The objective:

> Given a research question, search for relevant information, retrieve sources, analyze the results, and produce a concise report.

The architecture is:



A minimal tool interface might be:

```
tools = {
    "search": search_web,
    "retrieve": retrieve_document,
}
```

The model receives tool definitions and can select among them.

---

18. The Agent Runtime

A minimal implementation might look like:

```
def run_agent(question):

    state = {
        "question": question,
        "messages": [],
        "observations": [],
        "step_count": 0,
    }

    while state["step_count"] < 10:

        decision = llm(
            system_prompt=AGENT_PROMPT,
            state=state,
            tools=TOOLS,
        )

        if decision.type == "final":
            return decision.content

        if decision.type != "tool_call":
            raise RuntimeError("Invalid agent decision")

        tool = TOOLS[decision.tool]

        result = tool(**decision.arguments)

        state["observations"].append({
            "tool": decision.tool,
            "arguments": decision.arguments,
            "result": result,
        })

        state["step_count"] += 1
```

```
    return "Unable to complete research within the execution budget."
```

This is intentionally small.

The purpose is to expose the architecture, not hide it behind an agent framework.

---

19. The Agent Prompt

The model needs an explicit operational contract.

A useful system prompt might establish:

```
You are a research agent.

Your objective is to answer the user's question using
verifiable evidence.

You may:
1. Search for relevant sources.
2. Retrieve source contents.
3. Analyze retrieved information.
4. Search again when evidence is insufficient.
5. Produce a final report.

Do not invent evidence.

Prefer primary sources.

If evidence is insufficient, explicitly state the limitation.

Stop when:
- the question has been adequately answered, or
- additional searching is unlikely to improve the answer.

You have a maximum of 10 tool calls.
```

Notice that the prompt specifies behavior, but the runtime still enforces the ten-call limit.

This distinction is critical.

---

20. The Agent Trace

Suppose the question is:

> How does Apple's latest GPU architecture differ from earlier Apple Silicon GPUs?

The trace might look like:

```
STEP 1

Reasoning:
Need authoritative information about the latest GPU architecture.

Action:
search("Apple latest GPU architecture")

Observation:
Results include Apple technical documentation,
press coverage, and benchmark articles.
```

Then:

```
STEP 2

Action:
retrieve("Apple GPU architecture documentation")

Observation:
Primary-source documentation retrieved.
```

Then:

```
STEP 3

Reasoning:
The primary source describes architectural features
but does not provide historical comparison.

Action:
search("Apple GPU architecture previous generation comparison")
```

Then:

```
STEP 4

Observation:
Several technical sources describe differences.

Reasoning:
Enough evidence exists to construct a comparison.
```

Finally:

```
STEP 5

Action:
finalize_report(...)
```

This trace reveals something important:

The agent is performing information acquisition, not merely text generation.

---

21. Deliberately Breaking the Agent

A useful engineering exercise is to make the system fail.

Do not immediately build the perfect agent.

Instead, create failure modes and observe the system.

Failure 1: Tool returns an error

Make `search()` fail 30% of the time.

```
if random.random() < 0.30:
    raise SearchUnavailable()
```

Observe:

- Does the agent retry?
- Does it change strategy?
- Does it hallucinate a result?
- Does it terminate?
- Does it report the failure?

Failure 2: Empty results

Return:

```
[]
```

The agent must recognize that:

```
no results
```

does not mean:

```
the proposition is false
```

This is a fundamental distinction between absence of evidence and evidence of absence.

Failure 3: Malformed tool arguments

Force the model to occasionally produce:

```
{
  "query": null
}
```

The runtime should reject it before execution.

The agent should receive a structured error:

```
{
  "error": "invalid_arguments",
  "field": "query",
  "message": "query must be a string"
}
```

The model can then repair its action.

---

22. Failure 4: Tool Returns Misleading Data

This is particularly important.

Suppose the tool returns:

```
Source:
"XYZ technology provides a 3x performance improvement."
```

but the source is actually low-quality marketing material.

The agent must distinguish:

`tool result`

from:

`verified fact`

Tool access does not eliminate hallucination.

It changes the hallucination surface.

An agent can now hallucinate:

- what a tool returned
- what a source means
- whether a source is authoritative
- whether two sources refer to the same thing
- whether evidence supports a conclusion

Grounding therefore requires more than retrieval.

It requires **evidence evaluation**.

---

23. Failure 5: Infinite Loop

Create a tool that always returns:

```
No useful information found.
```

A poorly designed agent may repeatedly search.

```
search
 ↓
nothing
 ↓
search
 ↓
nothing
 ↓
search
 ↓
nothing
 ↓
...
```

The runtime should eventually terminate.

For example:

```
MAX_STEPS = 10

if step_count >= MAX_STEPS:
    return failure_report()
```

More sophisticated systems can detect semantic repetition:

```
if equivalent_action(previous_action, current_action):
    repeated_actions += 1
```

and terminate when the agent is stuck.

---

24. Failure 6: Contradictory Sources

Give the agent two sources:

```
Source A:
Performance = 100
```

```
Source B:
Performance = 130
```

Now the agent must not simply choose the more convenient value.

It should reason about:

- source authority
- publication date
- measurement methodology
- experimental conditions
- definitions
- whether the numbers are actually comparable

This is where agentic reasoning becomes significantly more interesting than simple retrieval-augmented generation.

---

25. Failure 7: Permission Violation

Add a destructive tool:

```
def delete_file(path):
    ...
```

Then give the agent access to it.

The correct architecture should prevent the model from executing arbitrary destructive operations.

For example:

```
def authorize(action, user):

    if action.tool == "delete_file":
        return False

    return True
```

The important experiment is to try to defeat the policy using prompting.

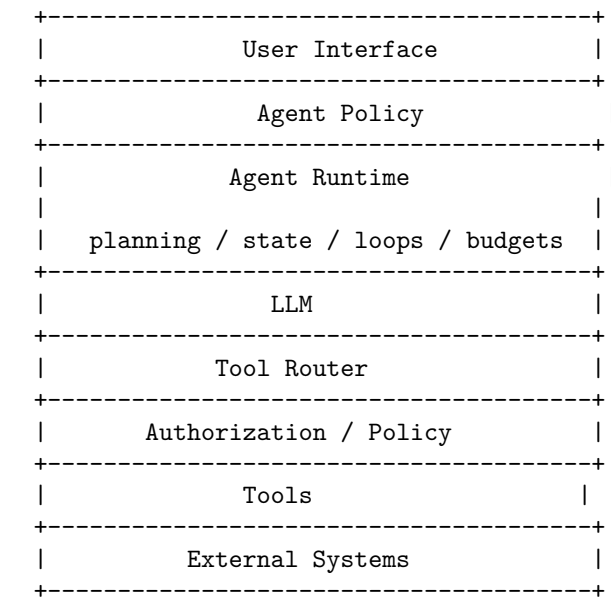
If the model can persuade the runtime to bypass authorization, the system is architecturally broken.

---

26. Agent Reliability Is a Systems Problem

At this point it should be clear why “agent engineering” is substantially more than prompt engineering.

A production agent contains at least these layers:



Each layer has different failure modes.

The LLM can:

- misunderstand the task
- hallucinate
- choose the wrong tool
- produce invalid arguments
- stop too early
- fail to stop

The runtime can:

- lose state
- exceed budgets
- mishandle retries
- incorrectly propagate errors

The tools can:

- timeout
- return incorrect information
- become unavailable
- change their API

The external systems can:

- reject requests
- change state

- become inconsistent
- impose rate limits

The engineering problem is therefore one of **composing unreliable components into a bounded and observable system**.

---

27. Observability

Agent systems require much richer observability than ordinary applications.

At minimum, record:

```
Run ID
User request
Model
Model parameters
Prompt/version
Step number
State snapshot
Model decision
Tool name
Tool arguments
Tool latency
Tool result
Token usage
Cost
Errors
Retries
Termination reason
Final result
```

A useful trace might look like:

```
RUN 8f31

STEP 0
  model: reasoning-model-vX
  tokens: 1,842
  decision: search
  query: "..."

STEP 1
  tool: search
  latency: 412 ms
  results: 8

STEP 2
```

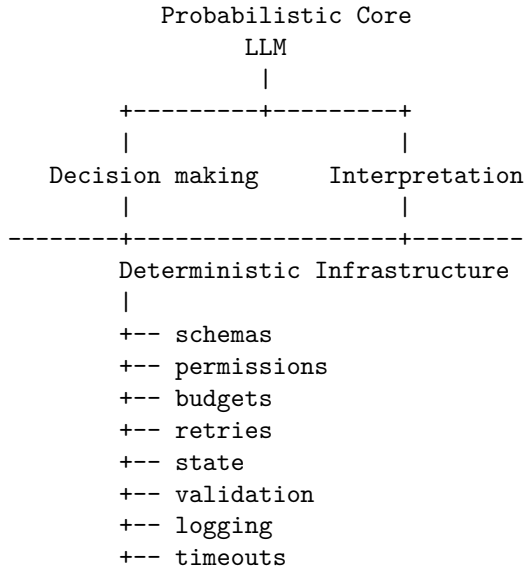
```
  decision: retrieve
  document: source_3

STEP 3
  tool: retrieve
  latency: 181 ms
  tokens_returned: 6,204

STEP 4
  decision: final

TERMINATION
  reason: goal_complete

TOTAL
  steps: 4
  latency: 7.2 s
  tokens: 11,823
  cost: $0.08
```

Without this information, debugging an agent becomes extremely difficult.

---

28. Deterministic Infrastructure Around a Probabilistic Core

A useful mental model is:



```
        +-- termination
```

The closer a component is to controlling external side effects, the more deterministic it should be.

This leads to a powerful design principle:

> **Put probabilistic behavior where judgment is valuable; put deterministic mechanisms around it where correctness is required.**

The model is good at:

- interpreting natural language
- selecting among strategies
- synthesizing information
- generating hypotheses
- deciding what information might be useful

Traditional software is better at:

- authorization
- schema validation
- accounting
- transactions
- resource limits
- retries
- state persistence
- timeouts
- access control

The strongest agentic architectures exploit both.

---

29. Agentic Workflow vs. Autonomous Agent

It is useful to distinguish two ends of the spectrum.

Workflow

```
Developer defines control flow
             ↓
       LLM fills in steps
```

Advantages:

- predictable
- testable
- observable
- easier to secure
- easier to debug

Autonomous agent

```
Developer defines objective + constraints
             ↓
        LLM determines
        execution path
```

Advantages:

- flexible
- handles unexpected situations
- can dynamically explore
- can adapt to changing information

Disadvantages:

- less predictable
- harder to evaluate
- harder to reproduce
- potentially more expensive
- more difficult to secure

The practical engineering answer is usually not to choose one universally.

Instead:

> **Use the least amount of autonomy required by the task.**

If the execution graph is known, use a workflow.

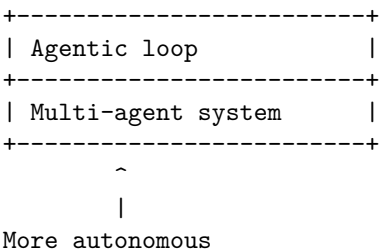
If the system genuinely needs to choose its own path through an uncertain environment, introduce agentic control.

---

30. The Agent Design Spectrum

This produces a useful spectrum:

```
More deterministic
        |
        v
+-------------------------+
| Single LLM call         |
+-------------------------+
| Structured LLM call     |
+-------------------------+
| LLM + fixed tools       |
+-------------------------+
| Conditional workflow    |
+-------------------------+
| Planner + workflow      |
```



Moving downward increases flexibility.

It also increases the system's state space and failure surface.

That tradeoff should be explicit.

---

31. A More Robust Agent Runtime

A production-oriented runtime might therefore look more like:

```python
def run_agent(goal, context):

    state = initialize_state(goal, context)

    while True:

        if budget_exceeded(state):
            return terminate(state, "budget_exceeded")

        if timeout_exceeded(state):
            return terminate(state, "timeout")

        if step_limit_exceeded(state):
            return terminate(state, "step_limit")

        decision = model_decide(state)

        if decision.type == "final":
            if validate_final_answer(decision, state):
                return decision.answer

            state = add_error(
                state,
                "Final answer failed validation"
            )
            continue
```

```
        if not valid_tool_call(decision):
            state = add_error(
                state,
                "Invalid tool call"
            )
            continue

        if not authorized(decision, state):
            state = add_error(
                state,
                "Tool call not authorized"
            )
            continue

        result = execute_with_retry(
            decision,
            state
        )

        state = update_state(
            state,
            decision,
            result
        )
```

The LLM remains important.

But it is only one component in the runtime.

---

32. The Core Engineering Insight

The progression from:

```
Prompt
 ↓
LLM
```

to:

```
Planner
 ↓
Tool
 ↓
Observation
 ↓
Reasoning
 ↓
```

```
Tool
 ↓
...
```

is not merely an increase in the number of model calls.

It represents a fundamental change in the computational model.

A conventional LLM application resembles:

$$y = f(x)$$

An agent resembles:

$$a_t \sim \pi_\theta(s_t)$$

$$o_t = T(s_t, a_t)$$

$$s_{t+1} = U(s_t, a_t, o_t)$$

until:

$$C(s_t) = \text{true}$$

where C is a termination condition.

The system has acquired:

- state
- actions
- observations
- feedback
- iteration
- memory
- constraints
- failure modes

In other words, it has acquired a **control loop**.

That is why agentic systems should be designed using ideas from both AI and systems engineering.

---

33. Engineering Principles

Several principles emerge from this architecture.

1. Treat tools as APIs Tool definitions should be explicit, typed, validated, and versioned.

2. Keep authorization outside the model The model can request an action. It cannot grant itself permission to execute it.

3. Make state explicit Do not allow critical state to exist only implicitly inside a prompt.

4. Bound everything Set limits on:

- steps
- tokens
- latency
- money
- tool calls
- retries
- context size

5. Design failure paths first Ask:

> What happens when this fails?

for every tool and every transition.

6. Make observations distinguishable from conclusions A retrieved fact and the model's interpretation of that fact are not equivalent.

7. Prefer workflows when possible Do not introduce autonomy simply because the technology makes it possible.

8. Instrument every step An agent without traces is extremely difficult to debug.

9. Evaluate the loop, not just the final answer Measure:

- tool selection
- argument correctness
- unnecessary calls
- recovery behavior
- termination
- cost
- latency
- final answer quality

10. Assume the model will eventually do something unexpected The runtime must remain safe when it does.

---

34. Chapter 5 Laboratory

The practical exercise for this day is deliberately small.

Build a research agent with exactly two tools:

```
search(query)
retrieve(document_id)
```

The agent should:

1. accept a research question;
2. search for relevant information;
3. retrieve useful sources;
4. inspect the retrieved evidence;
5. decide whether additional searching is necessary;
6. produce a final report.

Start with the simplest possible implementation.

Then introduce the following failures one at a time:

```
1. Search timeout
2. Empty search results
3. Malformed tool arguments
4. Retrieval failure
5. Duplicate searches
6. Contradictory sources
7. Low-quality sources
8. Infinite-loop behavior
9. Maximum-step exhaustion
10. Unauthorized tool call
```

For each failure, answer four questions:

```
What did the model do?

What did the runtime do?

What should have happened?

What instrumentation would have exposed the problem?
```

Then add:

```
- retry policies
- explicit state
```

```
- step limits
- cost limits
- tool validation
- authorization
- structured errors
- execution traces
- final-answer validation
```

The objective is not to build a sophisticated agent framework.

The objective is to understand what the framework is actually doing for you.

---

35. What You Should Understand by the End of Chapter 5

By the end of this day, you should be able to look at an agentic application and decompose it into:

```
Goal
 ↓
State
 ↓
Model decision
 ↓
Action
 ↓
Authorization
 ↓
Tool execution
 ↓
Observation
 ↓
State update
 ↓
Next decision
 ↓
...
 ↓
Termination
```

You should also be able to identify where each class of responsibility belongs.

```
LLM
    judgment
    interpretation
    planning
    synthesis
```

```
Runtime
    state
    loops
    budgets
    retries
    termination

Policy layer
    permissions
    safety
    authorization

Tools
    deterministic operations
    external effects

Observability
    traces
    metrics
    debugging
    evaluation
```

This separation is the foundation of reliable agentic engineering.

The important conceptual shift is therefore not:

> **"LLMs can now use tools."**

It is:

> **"We can embed a probabilistic policy inside a controlled software loop that can observe the environment and take actions."**

Once that loop exists, the engineering problem changes.

You are no longer primarily building prompts.

You are building a **stateful, partially autonomous software system whose control policy happens to be implemented by a probabilistic model**.

And that leads directly to the next problem: if the system can take multiple actions, recover from failures, and pursue goals autonomously, **how do we know whether it is actually working?**

That is the subject of evaluation.

---

36. Key Takeaways

1. **An agent is a control loop, not a single model call.** It repeatedly plans, acts, observes, and updates state until a termination condition is met.
2. **Prefer workflows over autonomy.** Use deterministic workflows when they suffice; add autonomy only where judgment or recovery genuinely helps.
3. **Treat tools as APIs.** Tool definitions should be explicit, typed, validated, and versioned.
4. **Keep authorization outside the model.** The model may request an action, but it cannot grant itself permission to execute it.
5. **Make state explicit.** Do not allow critical state to exist only implicitly inside a prompt.
6. **Separate observation from reasoning.** A retrieved fact is not the same as the model's interpretation of it.
7. **Bound every loop.** Set explicit limits on steps, tokens, latency, cost, retries, and context size so the system always terminates.
8. **Design failure paths first.** For every tool and every transition, specify what the runtime does when something goes wrong.
9. **Instrument every step.** An agent without traces is very hard to debug; log tool selection, recovery, and termination.
10. **Evaluate the loop, not just the answer.** Tool choice, argument correctness, recovery behavior, and final-answer quality all matter.

The engineering shift is therefore:

> **We no longer build only prompts. We build the deterministic boundaries—a bounded, authorized, observable, recoverable loop—around a probabilistic policy.**